PREDICTIVE MODELING USING MACHINE LEARNING

A Supervised Learning Project for Predicting Student Pass/Fail Outcomes

Project Type	Predictive Modeling using Machine Learning
Machine Learning Type	Supervised Learning – Classification
Algorithm Used	Random Forest Classifier
Programming Language	Python
Libraries	Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn

1. AIM

To build a machine learning model that predicts whether a student is likely to Pass or Fail based on study hours, attendance, previous marks, and assignment score.

2. OBJECTIVES

• Collect and prepare a small structured dataset for prediction.

• Handle the input features and target variable correctly.

• Split the dataset into training and testing data.

• Train a Random Forest classification model.

• Measure model performance using accuracy, precision, recall, and F1-score.

• Visualize the results using a confusion matrix and feature importance chart.

3. PROBLEM STATEMENT

Educational institutions can use historical student information to identify students who may need additional academic support. In this project, machine learning is used to predict the final outcome (Pass or Fail) from measurable academic features. The model learns patterns from training data and then predicts the outcome for unseen test data.

4. DATASET DESCRIPTION

The project uses a simple student-performance dataset. Each row represents one student.

Feature	Description	Type
Study_Hours	Hours studied per day	Numeric
Attendance	Attendance percentage	Numeric
Previous_Marks	Previous examination marks	Numeric
Assignment_Score	Assignment score	Numeric

Feature	Description	Type
Result	Final outcome: Pass or Fail	Target

5. METHODOLOGY

1. Create/load the dataset.
2. Separate input features (X) and target (y).
3. Split the data into 80% training and 20% testing sets.
4. Train the Random Forest Classifier using the training data.
5. Predict the outcomes for the test data.
6. Calculate evaluation metrics.
7. Display the confusion matrix and feature importance.

6. PYTHON IMPLEMENTATION

```
# Predictive Modeling Using Machine Learning

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create sample dataset
data = {
    "Study_Hours": [2,3,5,6,1,4,7,2,8,5,3,6,1,7,4,8,2,5,6,3],
    "Attendance": [60,65,80,85,55,75,90,62,95,82,68,88,50,92,78,96,58,84,86,70],
    "Previous_Marks": [45,50,65,70,35,60,80,48,85,68,52,75,30,82,62,88,42,72,78,55],
    "Assignment_Score": [40,45,70,75,30,65,85,42,90,72,50,80,28,88,68,92,38,76,82,58],
    "Result": [0,0,1,1,0,1,1,0,1,1,0,1,0,1,1,1,0,1,1,0]
}

df = pd.DataFrame(data)

# Input and output
X = df[["Study_Hours", "Attendance",
        "Previous_Marks", "Assignment_Score"]]
y = df["Result"]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Create and train model
model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                           target_names=["Fail", "Pass"]))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Fail", "Pass"],
            yticklabels=["Fail", "Pass"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# Feature Importance
importance = pd.Series(
    model.feature_importances_, index=X.columns
).sort_values(ascending=False)

importance.plot(kind="bar", figsize=(7, 4))
plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.tight_layout()
plt.show()

# Example prediction
new_student = [[5, 85, 70, 75]]
prediction = model.predict(new_student)

if prediction[0] == 1:
    print("Predicted Result: PASS")
else:
```

```
        print("Predicted Result: FAIL")
```

7. ALGORITHM USED – RANDOM FOREST

Random Forest is a supervised machine learning algorithm that combines many decision trees. Each tree gives a prediction, and the final class is selected from the combined tree predictions. It is useful for classification because it can capture non-linear relationships between input features and the target.

8. TRAINING AND TESTING

The dataset is divided into two parts: 80% for training and 20% for testing. The training set is used to learn patterns, while the test set is used to check how the trained model performs on unseen data. The random_state value makes the split reproducible.

9. MODEL EVALUATION

Metric	Meaning
Accuracy	Percentage of total predictions that are correct.
Precision	How many predicted positive cases are actually positive.
Recall	How many actual positive cases are correctly identified.
F1-Score	Balance between precision and recall.

10. CONFUSION MATRIX

The confusion matrix compares actual results with predicted results. It shows correct Pass predictions, correct Fail predictions, and incorrect predictions. The heatmap generated by the Python program provides an easy visual way to understand classification performance.

11. FEATURE IMPORTANCE

Random Forest can estimate the importance of each input feature. The feature-importance chart helps identify which student attributes contributed most to the model's predictions. The exact importance values are calculated when the code is executed.

12. SAMPLE OUTPUT

Output Item	Example
Model	Random Forest Classifier
Training Data	80%
Testing Data	20%
Prediction	PASS / FAIL
Evaluation	Accuracy, Precision, Recall, F1-score
Visualization	Confusion Matrix and Feature Importance

13. ADVANTAGES

• Easy to implement using Python and Scikit-learn.

• Can handle multiple input features.

• Random Forest reduces dependence on a single decision tree.

• Provides useful evaluation metrics.

• Feature importance helps explain which inputs affect predictions.

14. LIMITATIONS

• The sample dataset is small and is mainly for demonstrating the machine learning workflow.

• Real-world prediction requires a larger and reliable dataset.

• Model performance may change with different datasets.

• Predictions should support academic analysis and should not be treated as a final decision about a student.

15. FUTURE ENHANCEMENTS

• Use a larger real-world dataset.

• Compare Random Forest with Decision Tree, Logistic Regression, and other classifiers.

• Perform hyperparameter tuning to improve the model.

• Use cross-validation for more reliable evaluation.

• Build a simple dashboard to display predictions and performance.

16. CONCLUSION

This project demonstrates a complete supervised machine learning workflow. A Random Forest Classifier is trained using student academic features and used to predict Pass or Fail outcomes. The project also

17. RESULT

The Random Forest model was successfully trained and tested using the student performance dataset. It predicts whether a student will Pass or Fail using study hours, attendance, previous marks, and assignment score. The model is evaluated using Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix. The project successfully demonstrates a complete supervised machine learning workflow using Python and Scikit-learn.